

Week- 6

Backend Engineering

Pawan Dhanjani
Launchpad

Advanced Java

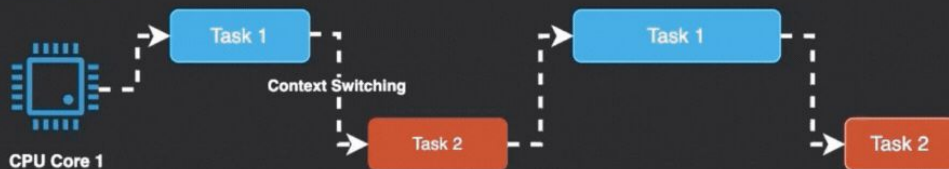
Agenda

- Java concurrency and multithreading
- Threads/Executor Service/Completable Future
- Java Memory management
- Garbage collection algorithms
- Types of garbage collection
- Question and answers

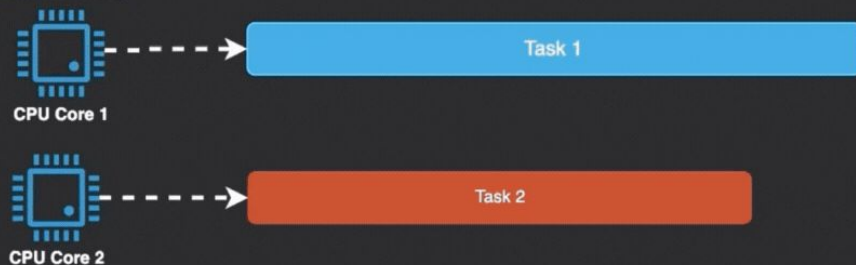
Concurrency VS Parallelism

CONCURRENCY VS PARALLELISM

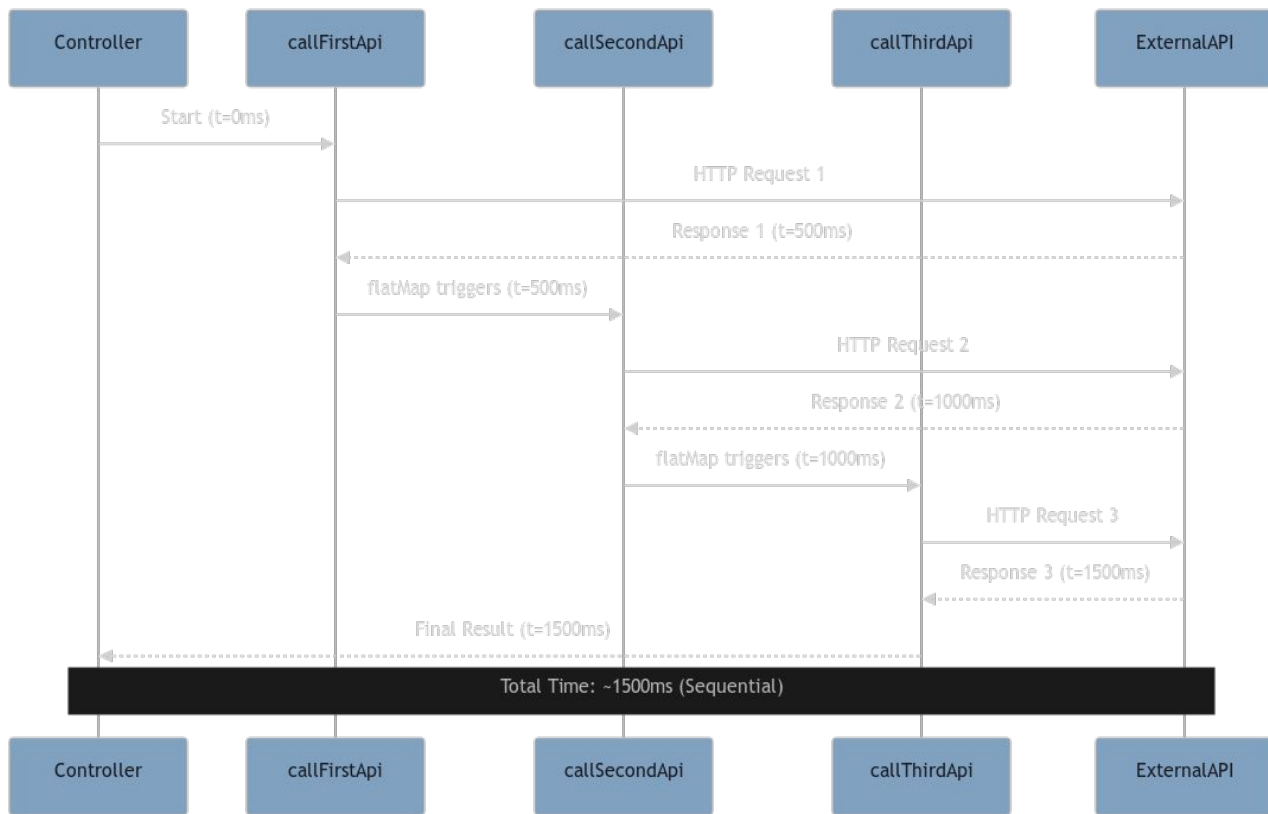
Concurrency



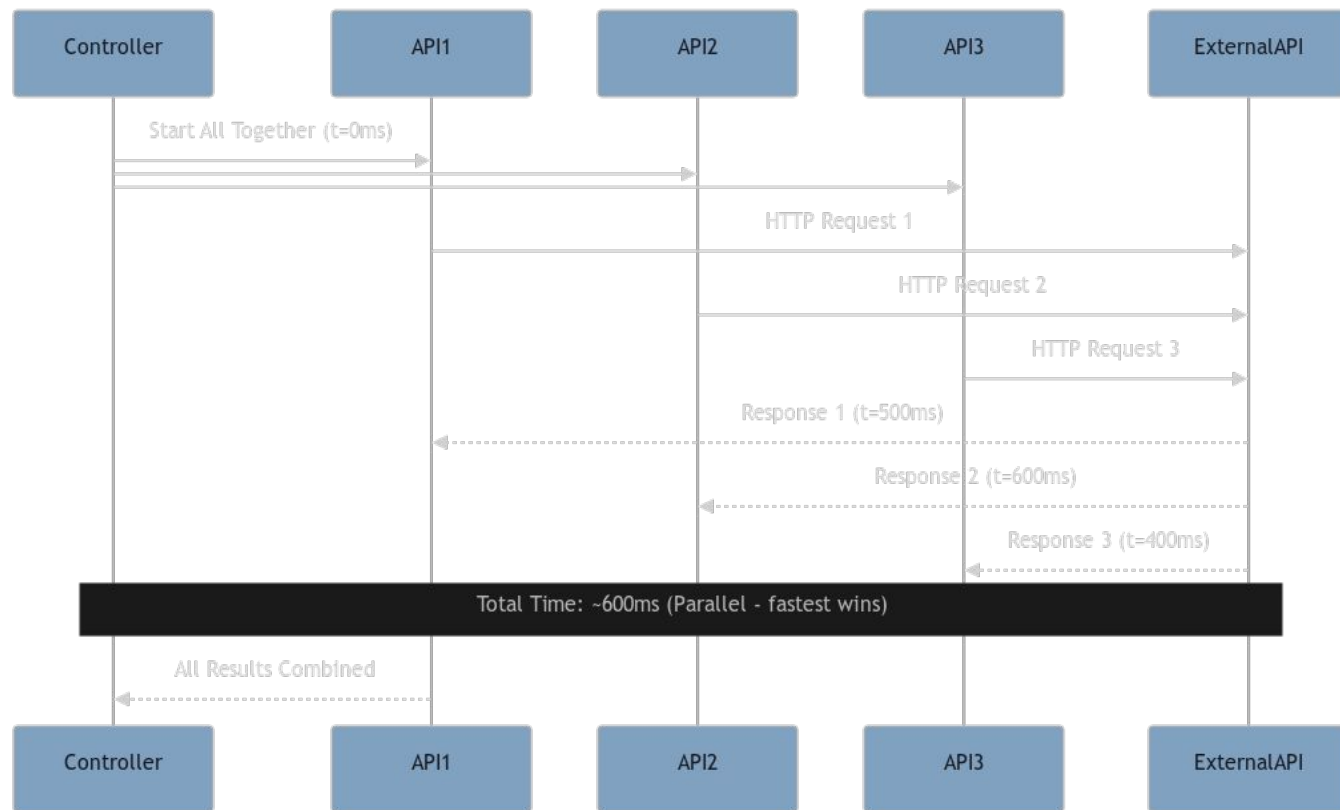
Parallelism



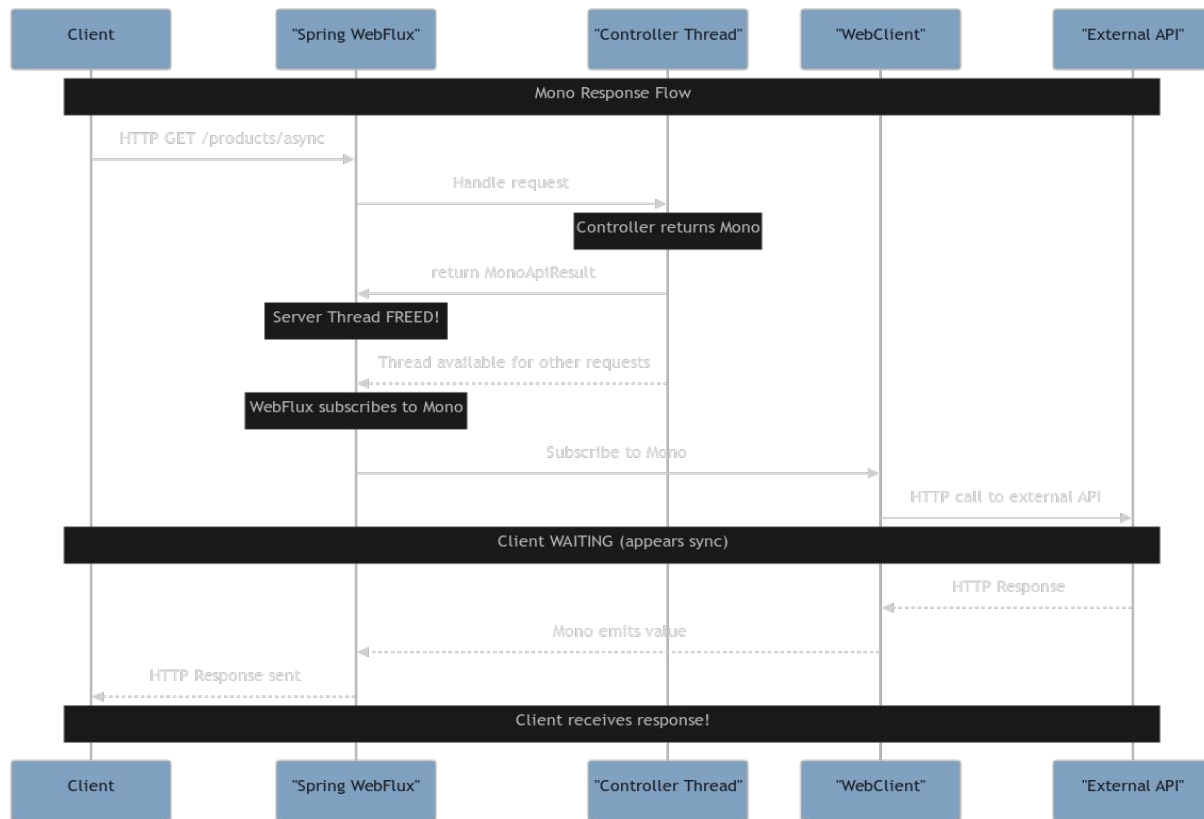
Sequential Asynchronous chaining



Parallel Invocation



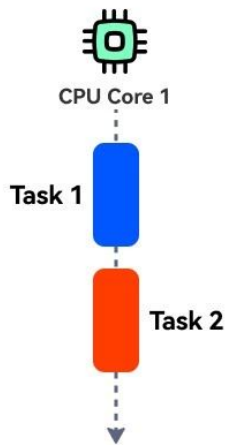
Async API Client flow



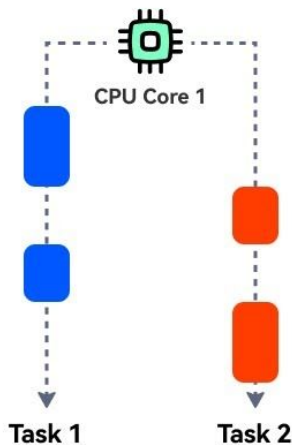
Concurrency VS Parallelism

Concurrency is **NOT** Parallelism

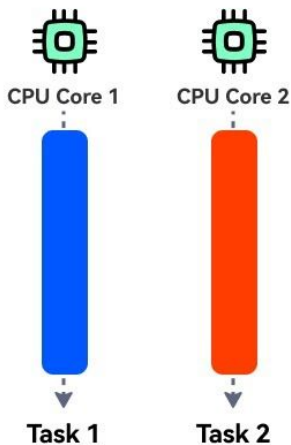
Not Concurrent,
Not Parallel



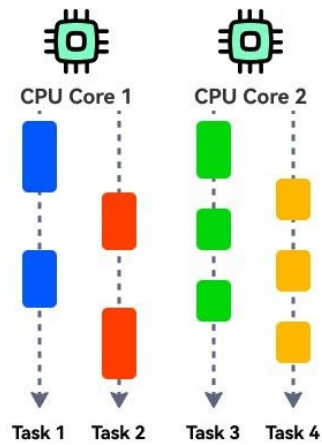
Concurrent
Not Parallel



Not Concurrent
Parallel



Concurrent
Parallel



What is concurrency ?

1. **What is Concurrency?** The ability of a program to make progress on multiple tasks simultaneously.
2. *What is multithreading - A feature in Java that allows multiple threads (lightweight sub-processes) to run concurrently.*
3. Multithreading is just one way to achieve concurrency in Java. Concurrency can also be achieved through other means, such as multiprocessing, asynchronous programming, or event-driven programming.

Multithreading V/S Multiprocessing

Parameter	Multiprocessing	Multithreading
Basic	Multiprocessing helps you to increase computing power.	Multithreading helps you to create computing threads of a single process to increase computing power.
Execution	It allows you to execute multiple processes concurrently.	Multiple threads of a single process are executed concurrently.
CPU switching	In Multiprocessing, CPU has to switch between multiple programs so that it looks like that multiple programs are running simultaneously.	In multithreading, CPU has to switch between multiple threads to make it appear that all threads are running simultaneously.
Creation	The creation of a process is slow and resource-specific.	The creation of a thread is economical in time and resource.
Classification	Multiprocessing can be symmetric or asymmetric.	Multithreading is not classified.
Memory	Multiprocessing allocates separate memory and resources for each process or program.	Multithreading threads belonging to the same process share the same memory and resources as that of the process.
Pickling objects	Multithreading avoids pickling.	Multiprocessing relies on pickling objects in memory to send to other processes.
Program	Multiprocessing system allows executing multiple programs and tasks.	Multithreading system executes multiple threads of the same or different processes.
Time taken	Less time is taken for job processing.	A moderate amount of time is taken for job processing.

Thread V/S Process

S.N.	Process	Thread
1	Process is heavy weight or resource intensive.	Thread is light weight, taking lesser resources than a process.
2	Process switching needs interaction with operating system.	Thread switching does not need to interact with operating system.
3	In multiple processing environments, each process executes the same code but has its own memory and file resources.	All threads can share same set of open files, child processes.
4	If one process is blocked, then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, a second thread in the same task can run.
5	Multiple processes without using threads use more resources.	Multiple threaded processes use fewer resources.
6	In multiple processes each process operates independently of the others.	One thread can read, write or change another thread's data.

What is a thread ?

1. 'thread' is a single stream of processes that can be executed independently by a computer's processor.
2. Why should we use threads in Java ?
 - a. Improved performance
 - b. Better resource utilization
 - c. Improved responsiveness
 - d. Robust concurrency API in Java

Concurrency drawbacks

1. Complexity
2. Debugging Difficulties
3. Resource Contention
4. Overhead
5. Complicated to test

How to create a thread in Java ?

How to create Thread in Java

Two ways to create thread in Java



- ➔ By extending Thread class
- ➔ By implementing Runnable Interface

Creating Threads

► By Inheriting from *Thread Class*

```
package org.airtribe.AsyncSyncApiCall.controller;  
  
2 usages  
public class ExampleThread extends Thread {  
    @Override  
    public void run() {  
        // contains all the code you want to execute  
        // when the thread starts  
  
        // prints out the name of the thread  
        // which is running the process  
        System.out.println(Thread.currentThread().getName());  
    }  
}
```


Trivia

- ▶ One common mistake is to call the `run()` method to start the thread. It might seem correct as everything works just fine, but calling the `run()` method does not start a new thread. Instead, it executes the code of the thread inside the parent thread. We use the `start()` method to execute a new thread.
- ▶ You can test this by calling `"thread.run()"` instead of `"thread.start()"` in the above code. You'll see that `"main"` is printed in the console, which means we are not creating any threads. Instead, the task is executed in the main thread.

Creating Threads

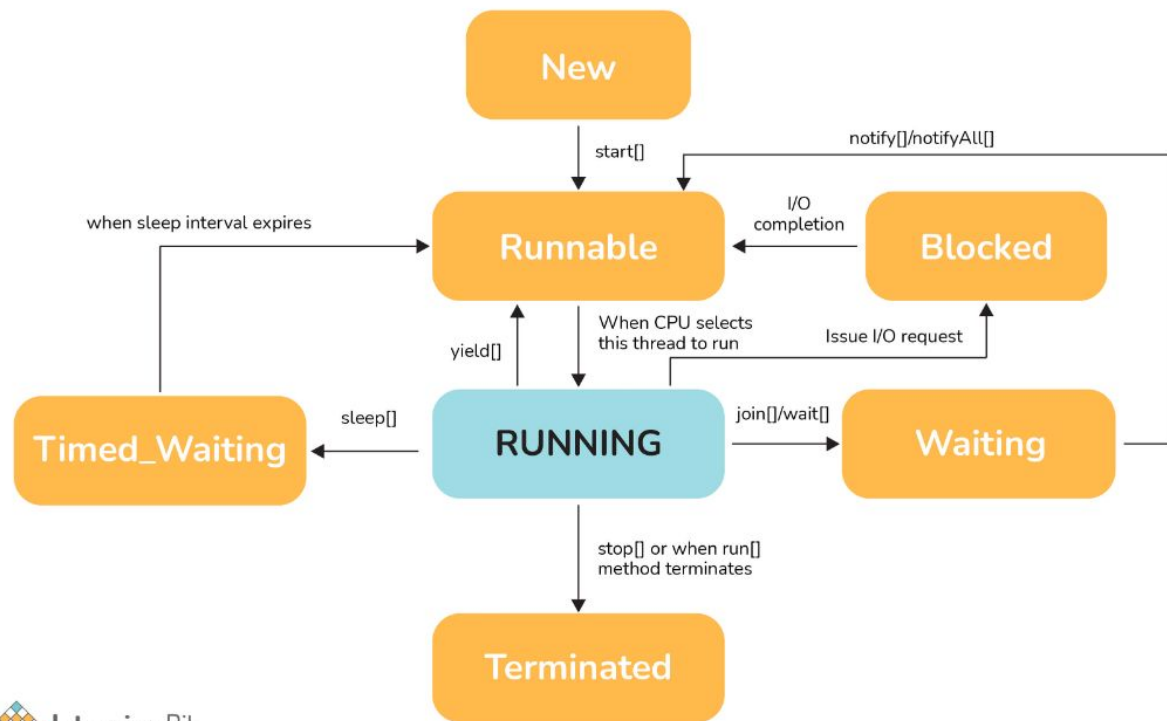
➤ By Implementing the ***Runnable*** Interface

➤ Both the methods work exactly the same with no difference in performance.

However, the Runnable interface leaves the option of extending the class with some other class since you can inherit only one class in Java.

```
package org.airtribe.AsyncSyncApiCall.controller;  
  
2 usages  
public class ExampleRunnable implements Runnable {  
    @Override  
    public void run() { System.out.println(Thread.currentThread().getName()); }  
}
```

Thread Lifecycle



Race conditions

- ▶ A **race condition** occurs when two or more threads access shared data and try to change it at the same time.
- ▶ The final outcome depends on the non-deterministic timing of the threads involved, leading to unpredictable results.
- ▶ Shared resources could be variables, data structures, files, or any other data that multiple threads might read/write.

Causes of Race Conditions

- ▶ **Lack of synchronization:** Threads modifying shared data without proper synchronization mechanisms (`synchronized`, `Lock`).
- ▶ **Improper use of volatile variables:** Misunderstanding the semantics of `volatile` in Java can lead to race conditions.
- ▶ **Non-atomic operations:** Operations that appear to be atomic but involve multiple steps, like `count = count + 1`.

Preventing Race conditions

- ▶ **Synchronization:** Use `synchronized` blocks or methods to ensure only one thread accesses a critical section at a time.
- ▶ **Locks:** Use `java.util.concurrent.locks.Lock` for more flexible and sophisticated locking mechanisms.
- ▶ **Atomic Variables:** Use `java.util.concurrent.atomic` classes (e.g., `AtomicInteger`, `AtomicReference`) for atomic operations on shared data.
- ▶ **Thread-Safe Collections:** Use thread-safe collections like `ConcurrentHashMap` or `CopyOnWriteArrayList`.

Thread Pool and Executor Service

- ▶ A **Thread Pool** is a group of pre-instantiated reusable threads that are available for performing tasks.
- ▶ Instead of creating a new thread for each task, the thread pool reuses existing threads, improving performance and resource management.

Why use thread pool ?

- ▶ **Improved Performance:** Reusing threads reduces the overhead of thread creation and destruction.
- ▶ **Efficient Resource Management:** Limits the number of threads running concurrently, preventing excessive resource consumption.
- ▶ **Better Control:** Provides control over the maximum number of concurrent threads, task scheduling, and execution.

Executor-Service

- ▶ **Executor Service** is a high-level replacement for working with threads directly.
- ▶ **Fixed Thread Pool**: A thread pool with a fixed number of threads.
- ▶ **Cached Thread Pool**: A thread pool that creates new threads as needed but reuses previously constructed threads when they are available.
- ▶ **Single Thread Executor**: An executor that uses a single worker thread.
- ▶ **Scheduled Thread Pool**: An executor that can schedule commands to run after a given delay or execute periodically.

Completable-Future

► **CompletableFuture** is a part of the `java.util.concurrent` package introduced in Java 8, designed to simplify asynchronous programming. It represents a future result of an asynchronous computation and provides a rich API for chaining tasks and handling results or exceptions.

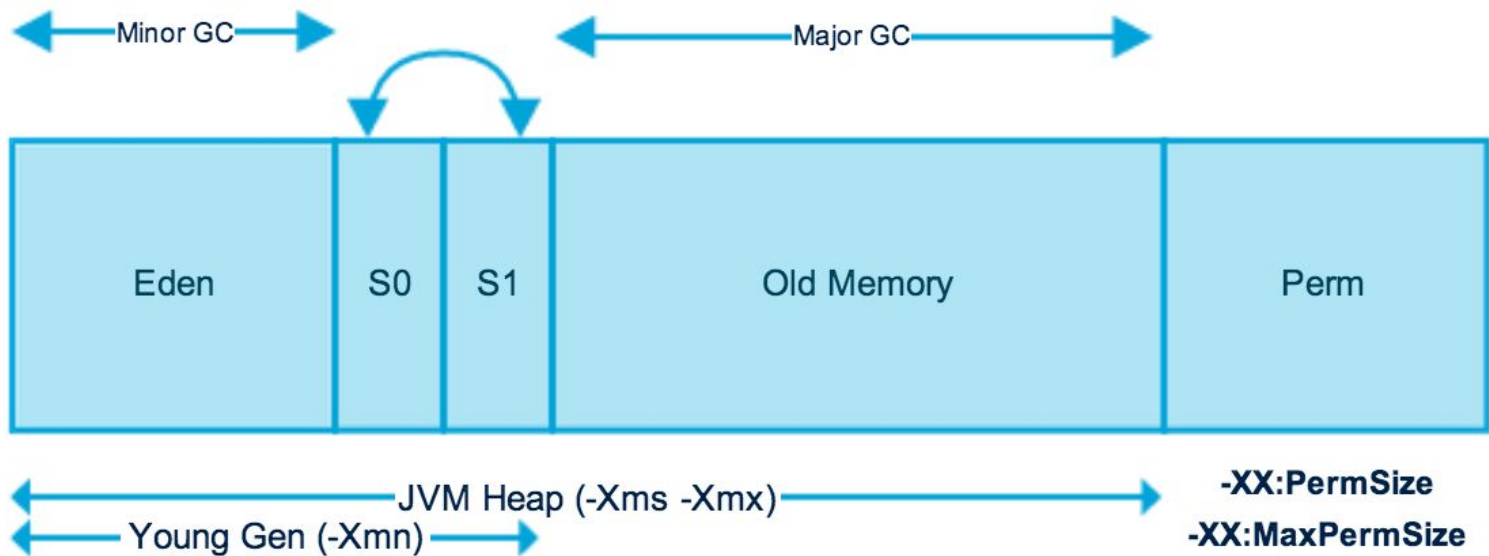
Key features of CompletableFuture

1. **Asynchronous Computation:**
 - Allows you to write non-blocking code by performing tasks asynchronously.
2. **Chaining:**
 - Supports chaining multiple stages of computation using `thenApply`, `thenAccept`, and `thenCompose`.
3. **Exception Handling:**
 - Provides methods to handle exceptions that occur during asynchronous computation.
4. **Combining Multiple Futures:**
 - You can combine multiple `CompletableFuture` instances using methods like `allOf`, `anyOf`, and `thenCombine`.
5. **Non-blocking:**
 - Methods such as `supplyAsync` and `runAsync` enable non-blocking operations, improving performance and responsiveness.

Garbage Collection in Java

- ▶ Garbage Collection (GC) is an automatic memory management process in Java.
- ▶ GC is performed to identify and reclaim memory occupied by objects that are no longer reachable by the application, thus preventing memory leaks and optimizing resource usage.

Java Memory Model



Java Memory Model

- **Heap Memory:**
 - **Young Generation:** Where new objects are allocated. Divided into Eden Space and Survivor Spaces (S0 and S1).
 - **Old Generation:** For long-lived objects. Involves Mark-Compact or Mark-Sweep algorithms for GC.
 - **Metaspace (Java 8+):** Replaces Permanent Generation, used for class metadata and dynamically allocated from native memory.

Java Memory Model

- **Non-Heap Memory:**
 - **Method Area:** Stores class structures and method data.
 - **Program Counter Register:** Holds the address of the current instruction being executed.
 - **Stack Memory:** Stores local variables, method calls, and return addresses.

Garbage Collection Algorithms

- **Mark-and-Sweep:**
 - **Mark Phase:** Identifies which objects are still in use.
 - **Sweep Phase:** Reclaims memory used by unreachable objects.
- **Copying Collection:**
 - **Young Generation:** Objects are copied from Eden to Survivor spaces.

Garbage Collection Algorithms

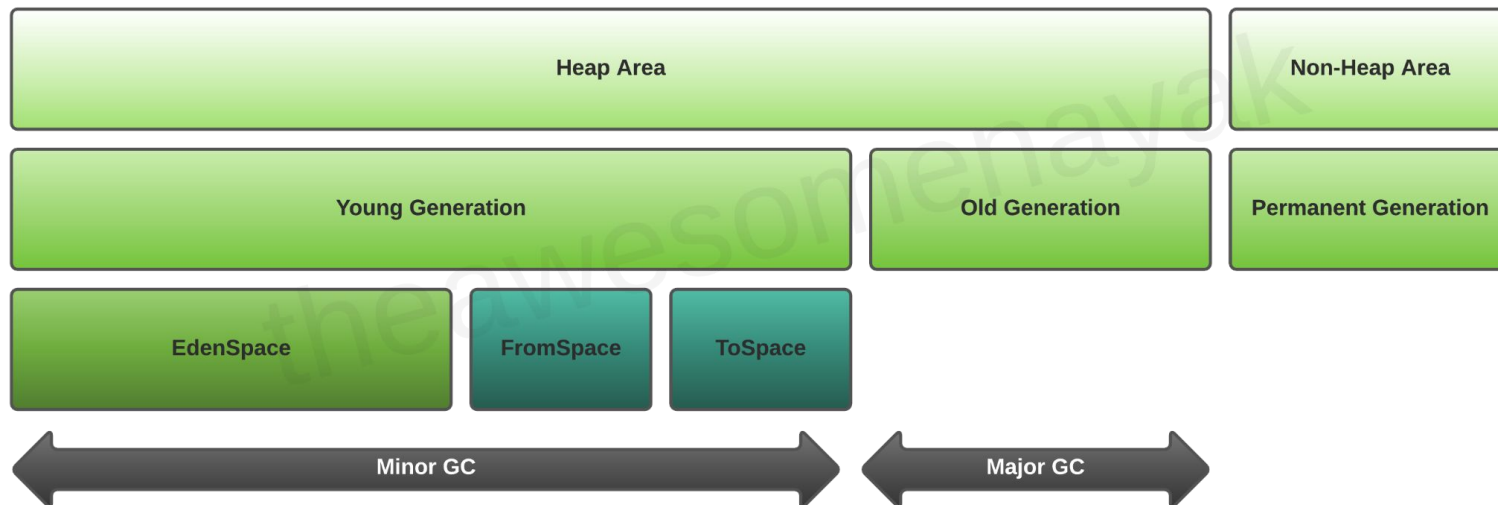
- **Mark-Compact:**

- Combines marking and compacting to reduce fragmentation in the Old Generation.

- **Generational Collection:**

- Optimizes GC by separating objects based on their age and frequency of collection.

GC Phases and Process



GC Phases and Process

Minor GC:

- Occurs in the Young Generation.
- Fast and typically involves copying live objects from Eden to Survivor spaces.

Major GC (Full GC):

- Occurs in the Old Generation.
- More comprehensive and involves marking and compacting.

Java Garbage Collector Types

► Serial GC:

- Single-threaded collector.
- Best for single-core systems or small applications.

► Parallel GC:

- Multi-threaded, designed for throughput.
- Suitable for multi-core systems.

Java Garbage Collector Types

► Concurrent Mark-Sweep (CMS) GC:

- Minimizes pause times by performing most work concurrently with the application.

► G1 GC (Garbage-First):

- Aims to balance between pause times and throughput.
- Divides heap into regions and collects them in a prioritized manner.

Serial Garbage Collectors

- ▶ This is the simplest GC implementation, as it basically works with a single thread. As a result, **this GC implementation freezes all application threads when it runs.**

Therefore, it's not a good idea to use it in multi-threaded applications, like server environments.

Parallel Garbage Collectors

- It's the default GC of the JVM from Java 5 until Java 8 and is sometimes called Throughput Collectors. Unlike Serial Garbage Collector, it uses multiple threads for managing heap space, but it also freezes other application threads while performing GC.

If we use this GC, we can specify maximum garbage collection threads and pause time, throughput, and footprint (heap size).

G1 Garbage Collectors

- *G1 (Garbage First) Garbage Collector* is designed for applications running on multi-processor machines with large memory space. It's available from the *JDK7 Update 4* and in later releases.

Unlike other collectors, the *G1* collector partitions the heap into a set of equal-sized heap regions, each a contiguous range of virtual memory. When performing garbage collections, *G1* shows a concurrent global marking phase (i.e. phase 1, known as *Marking*) to determine the liveness of objects throughout the heap.

G1 Garbage Collectors

- ▶ After the mark phase is complete, *G1* knows which regions are mostly empty. It collects in these areas first, which usually yields a significant amount of free space (i.e. phase 2, known as *Sweeping*). That's why this method of garbage collection is called Garbage-First.

Got Questions?

Thank you